MODELO MOBILENET V2

Tras observar que MobileNet V2 es uno de los mejores modelos de IA destinados a la clasificación de imágenes, vamos a tratar de entenderlo para posteriormente implementarlo.

MobileNet V2 es una red neuronal convolucional (CNN) de 53 capas, ligera y eficiente, diseñada por Google para dispositivos móviles y visión por computadora en el borde (edge computing). Utiliza convoluciones separables en profundidad y bloques residuales invertidos para equilibrar alta precisión y baja latencia, siendo ideal para clasificación de imágenes, detección de objetos y segmentación en entornos con recursos limitados. Esto es lo ideal para nuestro propósito.

Características más importantes:

- Arquitectura eficiente: Se compone de 53 capas profundas, diseñadas para ser rápidas y requerir menos potencia de procesamiento.
- Convoluciones separables en profundidad: Esta técnica clave reduce drásticamente el número de parámetros y cálculos en comparación con las CNN tradicionales, optimizando la velocidad sin sacrificar la precisión.
- Bloques residuales invertidos: Conecta capas de cuello de botella delgadas, mejorando la capacidad de representación de la red.
- Pre-entrenada con ImageNet: Viene con versiones preentrenadas capaces de clasificar imágenes en 1000 categorías diferentes (animales, objetos, etc.).
- Tamaño de Entrada: Diseñado principalmente para imágenes de 224x224 pixeles, aunque puede funcionar para imágenes de hasta 32x32 pixeles.
- Muchas aplicaciones: Muy utilizada en aplicaciones en tiempo real, análisis de imágenes médicas, reconocimiento de actividades humanas y detección de mascarillas.
- Modelo muy usado en TensorFlow Lite para hacer inferencias efectivas.

Ahora bien, trataremos de implementar nuestro modelo en especial. En primer lugar, generaremos nuestro dataset exactamente igual a como lo hemos hecho para modelos anteriores, la única diferencia será la resolución que será 224x224 y en lugar de blanco y negro será a color en formato RGB. En caso de que nuestro modelo se nos exceda de volumen esta será una de las cosas que podemos cambiar. También utilizaremos el aumento de datos por los motivos ya argumentados.

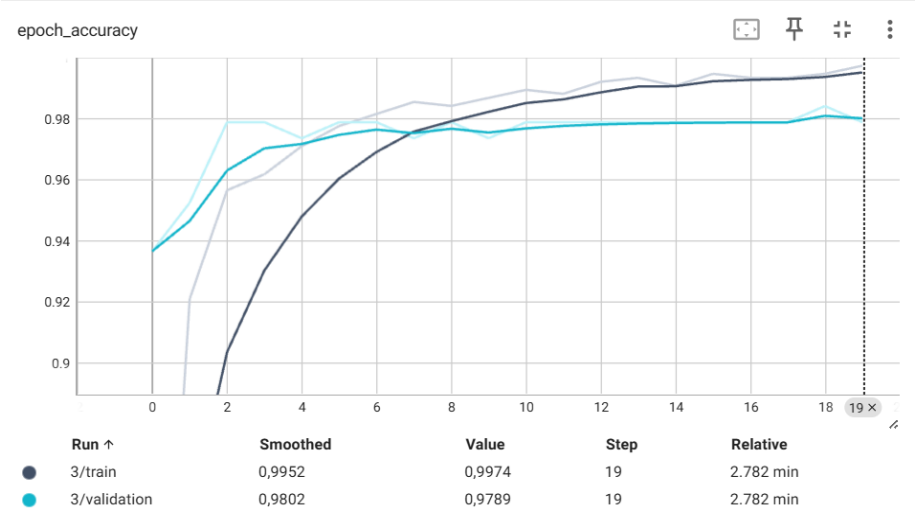
Ahora modificaremos el modelo para que nos sirva, entre las modificaciones es importante:

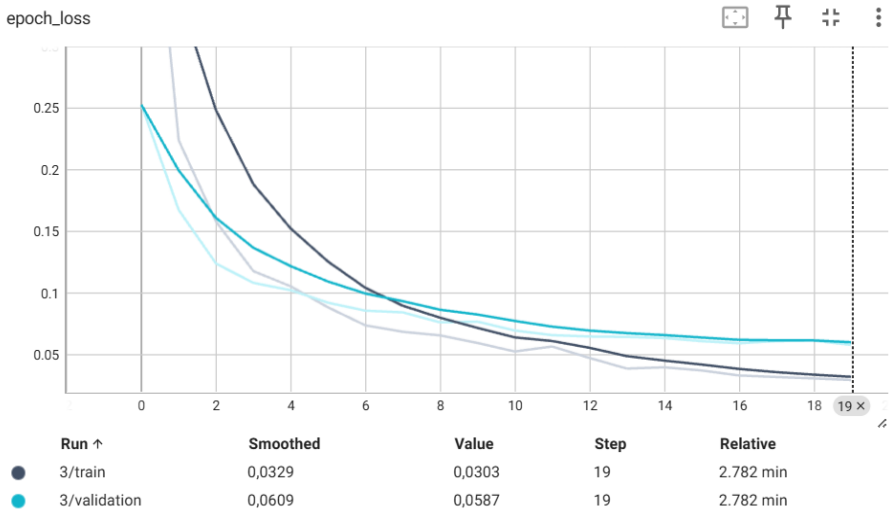
- alpha: este argumento determina el número de filtros que se utilizarán en la red neuronal. Si le damos un valor menor que 1, se reducirán con respecto al original, pero si el valor es mayor que 1, el número de filtros se verá aumentado. Como queremos un modelo ligero, utilizaremos un Alpha = 0.35.
- include_top: este argumento decide si la clasificación se hará con respecto a los 1000 elementos para los que ha sido entrenado con ImageNet. En nuestro caso solo queremos determinar si existe un caracol o no, por lo tanto, no será necesario incluir la última capa de la red en la que habrá una neurona por cada tipo de elemento. Entonces, include_top = False
- weights: Sirve para cargar el conocimiento de preentreno que nos interese. Esto es importante porque si analizamos ImageNet, encontramos que, entre los elementos, el número 113 se corresponde con “caracol” y nos será útil en el futuro. Por lo tanto, weights = “imagenet”.

110	flatworm, platyhelminth
111	nematode, nematode worm, roundworm
112	conch
113	snail
114	slug
115	sea slug, nudibranch
116	chiton, coat-of-mail shell, sea cradle, polyplacophore

Ahora que hemos personalizado nuestro MobileNet V2, lo utilizaremos en nuestro modelo. Para ello, crearemos una capa inicial para pasar la imagen, seguido de MobileNet V2 y para finalizar una capa que se encargue de clasificar el resultado. Esta capa se conformará de una capa de dropout para activar y desactivar ciertas neuronas de tal forma que estas traten de aprender distintas características, esto ya lo hemos explicado antes y finalmente una capa dense que determine un valor entre 1 y 0 en función de si predicción (1 si es caracol y 0 si no lo es).

Una vez hemos completado el modelo, lo que haremos es compilar y entrenarlo con el dataset preparado. Estos son los resultados obtenidos:





Estos resultados son muy buenos, casi perfectos.
Analizando lo que ha podido pasar para que sea tan bueno, lo que está ocurriendo es que se está aprovechando de sus conocimientos de preentreno donde ya sabe perfectamente distinguir un caracol. Entonces, lo que ha tenido que aprender el modelo es más simple, lo único que ha tenido que hacer es asociar que lo que queremos obtener es si encontramos su elemento 113 (el caracol) o no lo encontramos, por eso ha tardado solo unas 4 épocas en obtener tan buenos resultados.

Otra cosa interesante es que debido a la baja resolución y a la modificación de "alpha" generando que el modelo tenga menos filtros, hemos obtenido un modelo bastante pequeño. Las características son las siguiente:

Model: "sequential_9"

Layer (type)	Output Shape	Param #
rescaling_4 (Rescaling)	(None, 224, 224, 3)	0
mobilenetv2_0.35_224 (Functional)	(None, 7, 7, 1280)	410,208
global_average_pooling2d_7 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_6 (Dropout)	(None, 1280)	0
dense_7 (Dense)	(None, 1)	1,281

Total params: 414,053 (1.58 MB)
Trainable params: 1,281 (5.00 KB)
Non-trainable params: 410,208 (1.56 MB)
Optimizer params: 2,564 (10.02 KB)

Teniendo en cuenta que la capacidad de la memoria flash es de unos 2MB es un tamaño un poco elevado, pero todavía nos faltaría reducir el tamaño del modelo exportándolo a TensorFlow Lite, que será el próximo paso a seguir.